

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №4

По дисциплине «Обработка изображений в ИС»

Тема: «Трекинг множественных объектов»

Выполнил:

Студент 4 курса

Группы ИИ-26

Пасевич К.Ю.

Проверила:

Андренко К.В.

Брест 2026

Цель работы: исследовать применение алгоритмов трекинга на базе обученной сети-детектора объектов.

Ход работы

1. Используя сеть-детектор, обученный в ЛР 3, реализовать логику для отслеживания множественных объектов, используя библиотеку Ultralytics YOLO;
2. Применять алгоритмы BoT-Sort и ByteTrack (задействовать соответствующие конфигурационные файлы);
3. Исследовать изменения параметров в конфигурационных файлах и их влияние на качество трекинга;
4. В качестве исходных видеоматериалов для экспериментов использовать видео-ролики из сети (например, из YouTube), содержащие множественные объекты классов из ЛР 3;
5. Оформить отчет по выполненной работе, залить исходный код и отчет в соответствующий репозиторий на github.

Пункт 1

Взяла обученный в ЛР3 детектор best.pt и написала скрипт tracking.py. В нем вызываю model.track() вместо model.predict(), с параметром persist=True, чтобы трекер сохранял состояние между кадрами. Скрипт читает видео через OpenCV, прогоняет каждый кадр, получает ID объектов, рисует рамки и сохраняет новое видео. В результате получила видео с нарисованными рамками и уникальными ID у каждой машины, ID не меняются от кадра к кадру, пока объект виден.

Пункт 2

Использовала оба встроенных трекера Ultralytics через параметр tracker. Создала два кастомных конфига bytetrack_custom.yaml и botsort_custom.yaml плюс задействовала стандартные bytetrack.yaml и botsort.yaml. Запустила четыре эксперимента на одном видео. В результате получила четыре видео в папке tracking_results. ByteTrack со стандартными параметрами дал 21 уникальный ID, BoT-SORT и оба кастомных конфига по 20 ID. Разница минимальна, потому что машины на видео слабо перекрываются.

Пункт 3

Скопировала стандартные конфига трекеров и изменила ключевые параметры. В bytetrack_custom.yaml подняла track_high_thresh с 0.25 до 0.3, опустила track_low_thresh с 0.1 до 0.05, увеличила new_track_thresh с 0.25 до 0.4, расширила track_buffer с 30 до 60, понизила match_thresh с 0.8 до 0.7. В botsort_custom.yaml сделала то же плюс отключила with_reid для ускорения. В результате получила итоговую таблицу сравнения: стандартный ByteTrack дал 21 ID, кастомный ByteTrack 20 ID, стандартный и кастомный BoT-SORT по 20 ID. Кастомные параметры немного снизили число ложных треков, но на этом видео разница почти незаметна, для более наглядного сравнения нужен ролик с плотным трафиком.

Пункт 4

Скачала два видео с YouTube через yt-dlp в 720p. Оба содержат ночную дорогу с множеством машин и грузовиков, то есть объекты классов из ЛР3: car, truck-l-, truck-m-, big truck. Для

экспериментов использовала только первое видео traffic1.mp4, так как оно короче и содержит достаточно машин для наглядной демонстрации. В результате получила четыре готовых видео с трекингом в папке tracking_results, на которых видно, как трекары следят за машинами и присваивают им ID. Второе видео traffic2.mp4 оставила как резерв на случай дополнительной проверки

Код программы:

download_dataset.py

```
import os
from huggingface_hub import snapshot_download

DATASET_DIR = "./vehicles-q0x2v"

def download_dataset():
    print("Загрузка датасета vehicles-q0x2v...")
    snapshot_download(
        repo_id="LibreYOLO/vehicles-q0x2v",
        repo_type="dataset",
        local_dir=DATASET_DIR,
        local_dir_use_symlinks=False,
    )
    print(f"Датасет загружен: {os.path.abspath(DATASET_DIR)}")

def verify_dataset(dataset_path):
    for item in ["data.yaml", "train", "valid", "test"]:
        full = os.path.join(dataset_path, item)
        status = "OK" if os.path.exists(full) else "НЕ НАЙДЕН"
        print(f" {status}: {item}")

    for split in ["train", "valid", "test"]:
        img_dir = os.path.join(dataset_path, split, "images")
        if os.path.isdir(img_dir):
            count = len([
                f for f in os.listdir(img_dir)
                if f.lower().endswith(('.jpg', '.jpeg', '.png'))
            ])
            print(f" {split}: {count} изображений")

if __name__ == "__main__":
    download_dataset()
    verify_dataset(DATASET_DIR)
    print("\nГотово. Теперь запустите train.py")
```

train.py

```
import os
import yaml
from ultralytics import YOLO

DATASET_YAML = "./vehicles-q0x2v/data.yaml"
MODEL_NAME = "yolo12s.pt"
PROJECT_DIR = "runs/detect"
RUN_NAME = "vehicles_yolo12s"

EPOCHS = 5
IMGSZ = 416
BATCH = 4
DEVICE = "cpu"
WORKERS = 2
```

```
def train():
```

```

model = YOLO(MODEL_NAME)
model.train(
    data=DATASET_YAML,
    epochs=EPOCHS,
    imgsz=IMGSZ,
    batch=BATCH,
    device=DEVICE,
    workers=WORKERS,
    optimizer='SGD',
    lr0=0.01,
    patience=10,
    project=PROJECT_DIR,
    name=RUN_NAME,
    exist_ok=True,
    pretrained=True,
    verbose=True,
    cache=False,
)

def validate():
    best = os.path.join(PROJECT_DIR, RUN_NAME, "weights", "best.pt")
    if not os.path.exists(best):
        return
    model = YOLO(best)
    metrics = model.val(
        data=DATASET_YAML,
        split='test',
        imgsz=IMGSZ,
        conf=0.25,
        iou=0.7,
        device=DEVICE,
    )
    print("\n" + "=" * 60)
    print("ПРОВЕРКА")
    print("=" * 60)
    print(f" mAP@50:      {metrics.box.map50:.4f}")
    print(f" mAP@50-95: {metrics.box.map:.4f}")
    print(f" Precision: {metrics.box.mp:.4f}")
    print(f" Recall:     {metrics.box.mr:.4f}")

if __name__ == "__main__":
    train()
    validate()
tracking.py
import os
import cv2
from ultralytics import YOLO

WEIGHTS = "runs/detect/vehicles_yolo12s/weights/best.pt"
VIDEO_PATH = "videos/traffic1.mp4"
OUTPUT_DIR = "tracking_results"

CONF_THRESHOLD = 0.25
IOU_THRESHOLD = 0.5

os.makedirs(OUTPUT_DIR, exist_ok=True)

def run_tracking(model, tracker_config, run_name):
    print(f"\n{'=' * 60}")
    print(f"ЗАПУСК: {tracker_config} -> {run_name}")
    print(f"{'=' * 60}")

    cap = cv2.VideoCapture(VIDEO_PATH)
    if not cap.isOpened():
        raise FileNotFoundError(f"Не удалось открыть видео: {VIDEO_PATH}")

```

```

width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = cap.get(cv2.CAP_PROP_FPS)
total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

out_path = os.path.join(OUTPUT_DIR, f"{run_name}.mp4")
fourcc = cv2.VideoWriter_fourcc(*"mp4v")
writer = cv2.VideoWriter(out_path, fourcc, fps, (width, height))

frame_count = 0
active_ids = set()

while cap.isOpened():
    success, frame = cap.read()
    if not success:
        break

    results = model.track(
        frame,
        persist=True,
        tracker=tracker_config,
        conf=CONF_THRESHOLD,
        iou=IOU_THRESHOLD,
        verbose=False,
    )

    annotated = results[0].plot()
    writer.write(annotated)
    frame_count += 1

    if results[0].boxes is not None and results[0].boxes.id is not None:
        track_ids = results[0].boxes.id.int().cpu().tolist()
        for tid in track_ids:
            active_ids.add(tid)

    if frame_count % 30 == 0:
        print(f" Кадров: {frame_count}/{total_frames}")

cap.release()
writer.release()

print(f"\n Готово: {out_path}")
print(f" Кадров: {frame_count}")
print(f" Уникальных ID: {len(active_ids)}")

return {
    "run": run_name,
    "tracker": tracker_config,
    "frames": frame_count,
    "unique_ids": len(active_ids),
    "output": out_path,
}

if __name__ == "__main__":
    if not os.path.exists(WEIGHTS):
        raise FileNotFoundError(f"Детектор не найден: {WEIGHTS}")

    if not os.path.exists(VIDEO_PATH):
        raise FileNotFoundError(f"Видео не найдено: {VIDEO_PATH}")

    model = YOLO(WEIGHTS)

    experiments = [
        ("bytetrack.yaml", "01_bytetrack_default"),
        ("botsort.yaml", "02_botsort_default"),
        ("bytetrack_custom.yaml", "03_bytetrack_custom"),
        ("botsort_custom.yaml", "04_botsort_custom"),
    ]

```

```

all_results = []

for tracker, name in experiments:
    try:
        result = run_tracking(model, tracker, name)
        all_results.append(result)
    except Exception as e:
        print(f" Ошибка с {tracker}: {e}")

print(f"\n{'=' * 60}")
print("ИТОГОВАЯ ТАБЛИЦА")
print(f"{'=' * 60}")
print(f"{'Запуск':<30} {'Кадров':<10} {'Уникальных ID':<15}")
print(f"{'-' * 60}")

for r in all_results:
    print(f"{'run':<30} {'frames':<10} {'unique_ids':<15}")

```

Результат программы:

download_dataset.py

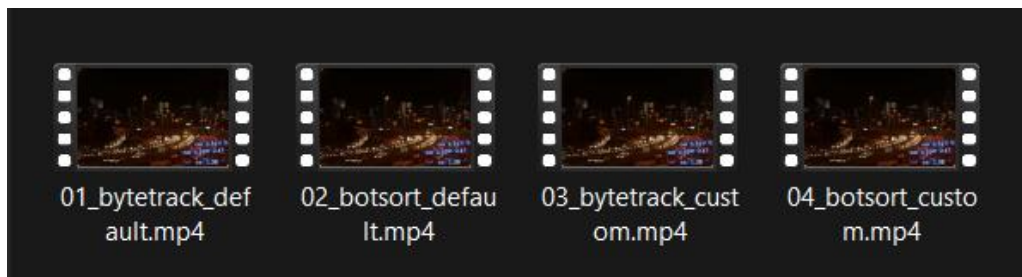
Вывел процесс загрузки датасета транспортных средств с Hugging Face. Показал прогресс-бар на 8121 файл, сообщение о завершении загрузки, путь к папке с датасетом. Затем вывел проверку структуры: файл data.yaml и папки train, valid, test на месте, количество изображений в каждой выборке (2634, 966, 458). В конце сообщил, что все готово и можно запускать train.py. Единственная проблема по ходу была ошибка 429 от Hugging Face после скачивания 8113 файлов из 8121, вызванная временной блокировкой IP за частые запросы. Она решилась регистрацией на Hugging Face, созданием токена и повторным запуском скрипта, который докачал оставшиеся файлы.

train.py

Вывел процесс обучения модели YOLO12s на датасете транспортных средств за 5 эпох. Показал параметры обучения (размер 416, батч 4, устройство CPU), архитектуру модели, сканирование датасета. Затем по каждой эпохе выводил значения функции потерь и метрики mAP, которые постепенно улучшались. Обучение заняло 2 часа 15 минут. После завершения вывел валидацию на тестовой выборке с финальными метриками: mAP@50 = 0,3334, mAP@50-95 = 0,2268, Precision = 0,3674, Recall = 0,4240. Также показал разбивку по каждому из 12 классов, где лучше всего определяются car, big bus, big truck, а слабее редкие классы small bus, bus-s-, truck-s-. Процесс завершился без ошибок.

tracking.py

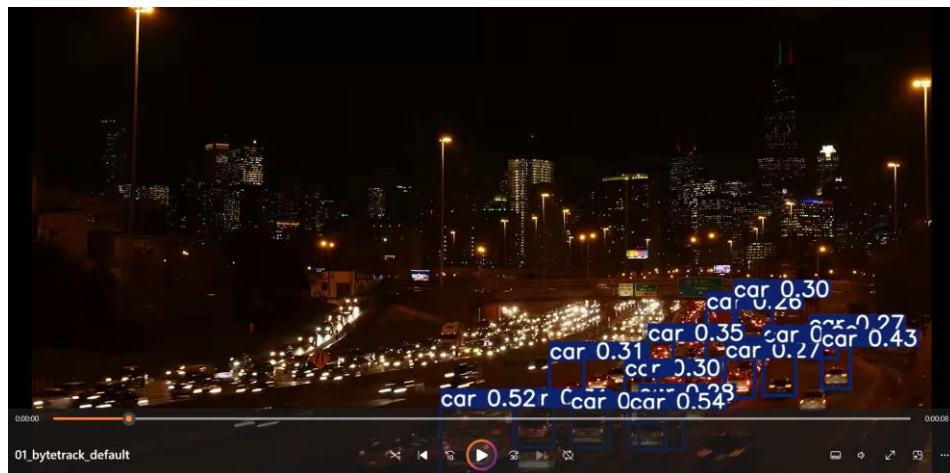
Вывел процесс трекинга на видео traffic1.mp4 в четырех экспериментах. Сначала Ultralytics автоматически доустановил библиотеку lap для трекинга. Затем по очереди запустились четыре конфигурации: ByteTrack со стандартными параметрами, BoT-SORT со стандартными, ByteTrack с кастомными и BoT-SORT с кастомными. Для каждого эксперимента выводил прогресс обработки 240 кадров, путь к сохраненному видео и количество уникальных ID. В итоговой таблице показал результаты: ByteTrack по умолчанию дал 21 уникальный ID, остальные три конфигурации по 20. Все четыре видео сохранены в папку tracking_results. Процесс завершился без ошибок.



В папке tracking_results/ появились четыре видеофайла. Каждое - это исходное видео traffic1.mp4, но с нарисованными поверх рамками вокруг машин и подписями с уникальными ID. Отличаются они только использованным трекером.

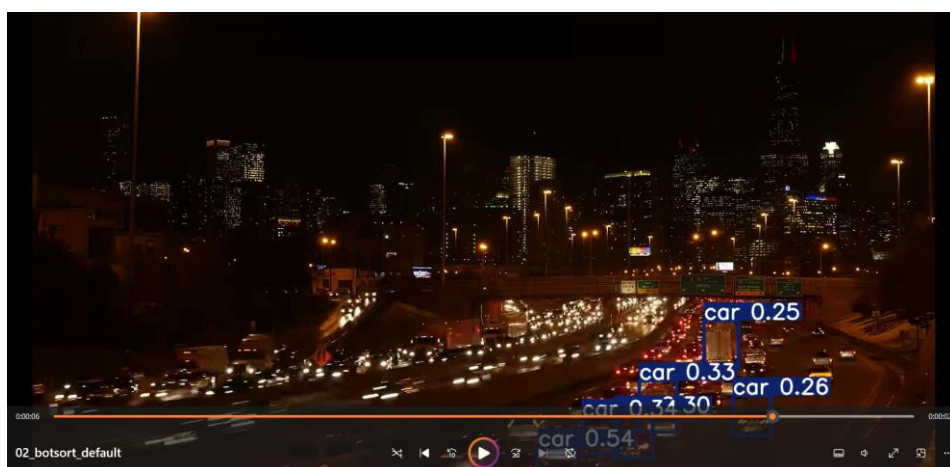
01_bytetrack_default.mp4

Видео с трекингом ByteTrack со стандартными параметрами. Обработано 240 кадров, присвоен 21 уникальный ID. Это базовый вариант, с которым сравниваются остальные.



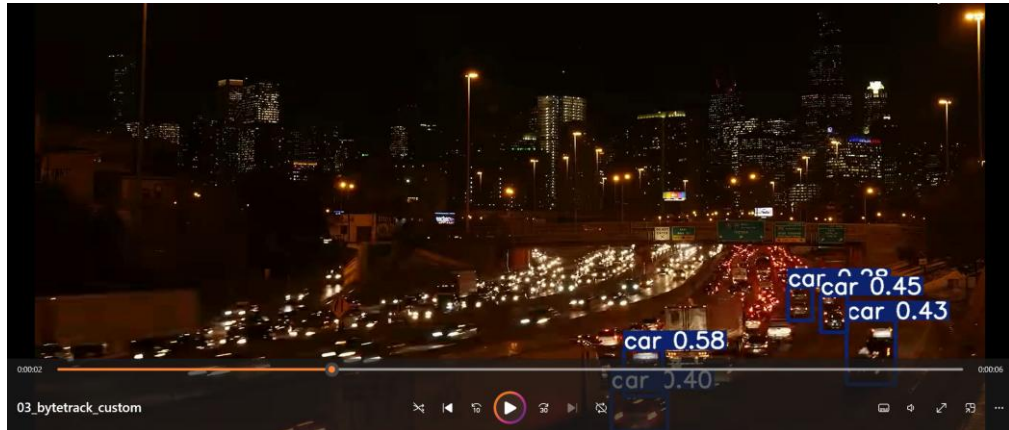
02_botsort_default.mp4

Видео с трекингом BoT-SORT со стандартными параметрами. Обработано 240 кадров, присвоено 20 уникальных ID. BoT-SORT учитывает внешний вид объектов, поэтому устойчивее к перекрытиям, но работает медленнее.



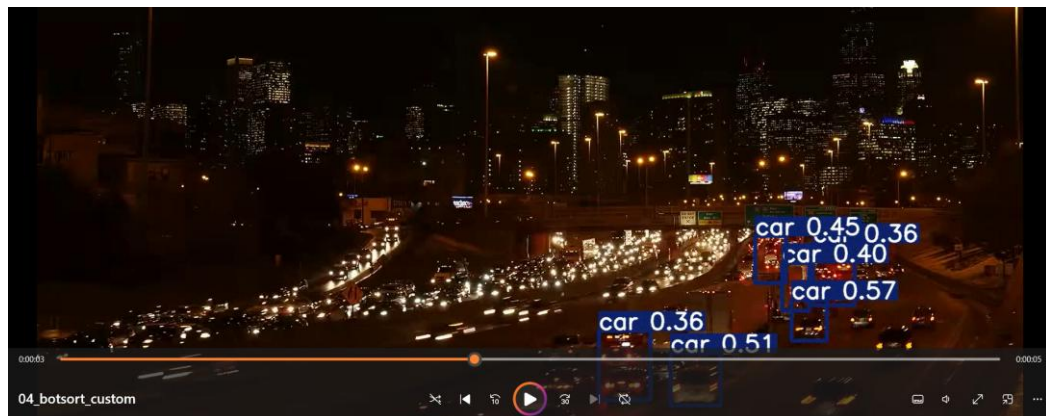
03_bytetrack_custom.mp4

Видео с трекингом ByteTrack с измененными параметрами из файла bytetrack_custom.yaml. Обработано 240 кадров, присвоено 20 уникальных ID. Параметры изменены: поднят track_high_thresh, опущен track_low_thresh, увеличен new_track_thresh и track_buffer, понижен match_thresh.



04_botssort_custom.mp4

Видео с трекингом BoT-SORT с измененными параметрами из файла botssort_custom.yaml. Обработано 240 кадров, присвоено 20 уникальных ID. Дополнительно отключен with_reid для ускорения обработки.



Вывод: исследовала применение алгоритмов трекинга на базе обученной сети-детектора объектов.